

Vionix 详细设计说明书

1. 后端分层

目标 Backend 使用 Spring Boot 4 分层：

```
backend/src/main/java/com/vionix/
├── VionixApplication.java
├── common/
│   ├── api/                # Result, PageResult, ErrorCode
│   ├── config/             # Web, Redis, InfluxDB, MQTT, Secret 配置
│   ├── security/           # SecurityContext, JWT, filters, annotations
│   ├── tenant/             # TenantContext, MyBatis tenant interceptor
│   └── audit/              # AuditLog, operation tracing
├── auth/                   # 登录、刷新、登出、token 记录
├── rbac/                   # 用户、角色、权限、数据范围
├── metrics/                # Influx 查询、bucket 路由、字段映射
├── mqtt/                   # MQTT consumer, payload parser
├── websocket/              # STOMP 鉴权、订阅校验、消息分发
├── rule/                   # 规则 CRUD、编译、运行时评估
├── alert/                  # 告警状态、查询、统计、推送
├── device/                 # 设备目录、设备分组和设备权限边界
└── dashboard/              # 仪表盘元数据、变量选项
```

2. 认证与授权设计

2.1 登录

？前端通过 HTTPS 提交用户名、密码、验证码和设备摘要。

？后端按租户上下文查找用户，校验密码 hash。

？登录成功后签发短期 accessToken，并通过 HttpOnly Cookie 下发 refreshToken。

？Redis 记录登录失败计数和必要的安全状态。

2.2 请求鉴权

HTTP Filter 解析 accessToken，构造：

字段	说明
userId	当前用户
tenantId	当前租户
roles	角色编码
permissions	API、菜单、数据权限
dataScope	SELF/GROUP/TENANT/ALL

Controller 使用注解声明权限，Service 层继续校验对象归属。

2.3 登出和刷新

场景	处理
refresh	校验 refreshToken hash、过期时间和设备信息后签发新 accessToken

场景	处理
logout	删除 refreshToken 记录，并把 accessToken jti 写入 Redis 黑名单
多实例	所有实例共享 Redis，保证登出和锁定状态一致

3. 租户隔离设计

存储	隔离方式
MySQL	业务表直接携带 tenant_id, MyBatis 插件追加过滤
InfluxDB	查询和写入均携带 tenant_id tag
Redis	key 中包含 tenantId 或明确全局命名空间
WebSocket	session 绑定 tenant_id, 订阅路径校验 tenant
MQTT	topic 或设备注册表补齐 tenant_id

服务层仍需做对象级校验，不能只依赖 SQL 拦截器。

4. 设备目录与权限设计

4.1 最小设备目录

设备目录用于建立 tenantid + deviceid

的权威归属关系。它不是完整设备注册中心，但必须支撑监控、规则、告警、仪表盘变量和 WebSocket 订阅的权限校验。

核心字段：

字段	说明
tenant_id	设备所属租户
device_id	租户内唯一设备标识，对应 MQTT topic 和 InfluxDB tag
name	展示名称
source	默认数据来源，如 mqtt、modbus、http
status	ENABLED、DISABLED、ARCHIVED
metadata	位置、型号、扩展属性等 JSON

4.2 设备权限服务

DeviceAccessService 提供统一能力：

方法	用途
assertVisible(tenantId, userId, deviceId)	历史查询、WS 订阅、规则目标校验
listVisibleDevices(tenantId, userId, filters)	监控页设备列表和仪表盘变量
assertGroupDevicesBelongToTenant(tenantId, groupId, deviceIds)	设备分组绑定校验
resolveDeviceTags(tenantId, deviceId)	补齐可选 tag，如 group_id、location

生产环境中，未登记或已禁用设备的指标写入应进入拒绝或隔离流程；单租户开发环境可临时自动登记，但自动登记设备默认不得跨租户可见。

5. 指标服务设计

5.1 写入模型

统一 MQTT topic:

```
vionix/{tenant_id}/{device_id}/metrics
```

payload 示例:

```
[json]
{
  "timestamp": "2026-05-26T10:01:00Z",
  "source": "mqtt",
  "temperature": 25.3,
  "humidity": 60.5
}
```

解析后写入 InfluxDB:

类别	字段
measurement	device_metrics
tags	tenantid, deviceid, source
fields	数值指标字段

生产环境写入前需要校验设备存在且启用。若 Telegraf 直接写入 InfluxDB，后端仍需在查询、WebSocket 和规则侧基于设备目录做权限校验；完整实现推荐由后端写入网关或 Telegraf tag 规则保证 tenantid 和 deviceid 正确。

5.2 查询路由

MetricsQueryService 根据时间跨度选择 bucket:

时间跨度	Bucket
<= 2m	device_raw
<= 2h	device_min
<= 90d	device_hour
> 90d	device_day

历史聚合字段由 metric + "_" + agg 生成。raw 层读取原始 field。

5.3 Flux 查询约束

- ? 必须包含 tenant_id tag 过滤。
- ? 如果传入 deviceid，必须先校验设备权限。
- ? 聚合层只能查询已定义的 mean/sum/max/min/*_count 字段。

? 用户传入字段需做白名单或合法字符校验，避免拼接风险。

6. WebSocket 设计

6.1 连接

推荐 STOMP over WebSocket:

```
CONNECT /ws
Authorization: Bearer {accessToken}
```

后端在 CONNECT 阶段校验 token，并把安全上下文绑定到 session。

6.2 订阅校验

Topic	校验
/topic/tenant/{tenantId}/metrics	tenantId 等于 session tenant，且有实时监控权限
/topic/tenant/{tenantId}/device/{deviceId}/metrics	tenantId 匹配，deviceId 在数据权限范围内
/topic/tenant/{tenantId}/alerts	tenantId 匹配，且有告警查看权限
/topic/tenant/{tenantId}/device/{deviceId}/alerts	tenantId 匹配，deviceId 授权

服务端只向匹配订阅发送数据，不允许广播全量后由前端过滤。

7. 规则引擎设计

7.1 运行时组件

组件	职责
RuleLoader	启动加载启用规则，规则变更后刷新缓存
RuleCompiler	将 DB 模型编译为运行时表达结构
RuleIndex	按 tenantid + scope + targetid 建立规则索引
StateTracker	按 tenantid + deviceid + metric 维护窗口状态
RuleEngine	对实时消息或巡检结果执行规则匹配
AlertManager	维护告警触发、恢复、抑制和升级状态
ActionDispatcher	异步执行规则动作并记录日志

7.2 条件语义

当前版本只支持：

```
(group_index = 0 的条件全部 AND)
OR
(group_index = 1 的条件全部 AND)
OR ...
```

rulecondition 不使用 logicop。后续如需嵌套表达式，新增条件树 JSON 或 AST 字段。

7.3 告警状态

状态	说明
FIRING	正在触发
RESOLVED	已恢复
SUPPRESSED	抑制期内
ESCALATED	已升级

同一规则、同一租户、同一设备在抑制窗口内不应重复生成等价告警。

8. 低代码仪表盘设计

仪表盘元数据存储在 MySQL，组件数据源复用已有监控接口：

数据类型	来源
历史指标	GET /api/metrics
实时指标	WebSocket /ws
告警列表	GET /api/alerts
变量选项	/api/dashboard-vars/*

layoutconfig 中允许存储变量、组件、布局和组件私有配置，不允许存储 tenantid 或服务端密钥。

9. 前端详细设计

前端按业务域组织：

```

frontend/src/
├── api/
├── router/
├── stores/
├── views/
│   ├── login/
│   ├── monitor/
│   ├── alerts/
│   ├── rules/
│   ├── dashboards/
│   ├── rbac/
│   └── tenants/
├── components/
└── utils/

```

前端职责：

- ？管理 accessToken 内存态和 refresh 流程。
- ？根据权限树渲染菜单和按钮。
- ？对 WebSocket 进行连接、重连、订阅和取消订阅。
- ？渲染历史图表和实时图表。
- ？编辑和渲染仪表盘布局。

安全职责仍以后端为准，前端权限隐藏不是安全边界。

10. 异常处理和日志

类别	处理
参数错误	返回 400 和字段级错误信息
未认证	返回 401，前端尝试刷新 token
无权限	返回 403，不暴露目标资源是否存在
资源不存在	返回 404
冲突	返回 409，如租户内名称重复
服务异常	返回 500，日志记录 traceId

所有入口日志应包含

traceId、tenantId、userId、API、耗时和结果，不记录密码、token、密钥和敏感字段明文。

必须写审计日志的事件：

事件	说明
跨租户操作	超级管理员指定 X-Tenant-Id 或切换目标租户
权限变更	用户角色、角色权限、数据范围调整
设备状态变更	设备启用、禁用、归档
规则变更	规则创建、更新、启停、删除
仪表盘发布	公开、取消公开、删除
外部动作配置	HTTP、MQTT、IM 动作配置新增或修改

审计日志记录摘要和结果，不记录密码、token、私钥、动作配置中的敏感 secret 明文。